Task 2: Real-Time Computer Vision Optimization

YOLOv8n Object Detection and Tracking

Prepared for execution in Google Colab or Kaggle GPU

Report status: Benchmark template — measured results have not yet been collected.

Project repository: https://github.com/kimiko-11/real-time-object-detection-tracking

Author: Kimaya Chavan

This report is designed to be completed after running the included benchmark. Do not present placeholder values as experimental findings.

1. Objective

Optimize the existing YOLOv8n detection and tracking pipeline for lower latency while measuring the effect on detection quality. The baseline is PyTorch FP32 at 640 × 640.

2. Baseline pipeline

The original program reads frames, invokes model.track() with persistent tracking, retains trajectory points, draws the trajectory and plots detection results. Repeatedly redrawing an unbounded trajectory can increase per-frame overhead.

3. Optimization strategies

FP16 inference: Use half precision on supported CUDA GPUs.
Input resolution: Compare 640 px with 512 px; lower resolution reduces spatial computation but may harm small-object detection.
Bounded trajectory history: Keep a fixed-size deque (30 points by default) to limit drawing and memory cost.
Benchmark discipline: Warm-up, CUDA synchronization, repeated measurements, and explicit environment logging.

4. Experimental protocol

Use the same GPU/runtime, confidence threshold, model weights, video, validation split and tracker settings for all variants. Use at least 100 timed iterations after 20 warm-up iterations and preferably repeat the experiment three times. COCO8 is a smoke-test dataset; a larger fixed validation set is needed for strong accuracy claims.

5. Results table

Fill the table with measured outputs from results/benchmark_results.csv. A dash means no result has been measured.

Variant	Size	Precision	Mean ms	FPS	mAP50	mAP50-95
Baseline	640	FP32	—	—	—	—
Optimized A	640	FP16	—	—	—	—
Optimized B	512	FP16	—	—	—	—

6. Metrics and calculations

FPS = 1000 / mean latency (ms). Speedup = baseline mean latency / optimized mean latency. mAP change = optimized mAP − baseline mAP. Report quality differences in percentage points when metrics are shown as percentages.

7. Trade-offs

FP16 may improve throughput on compatible GPUs but can introduce numerical differences. Lower input resolution can increase speed at the cost of detecting small or distant objects. Bounded trajectory history reduces rendering work but limits the visible past path.

8. Limitations

COCO8 is small and does not support strong generalization claims. Synthetic-image timing isolates model pipeline latency but does not represent real scenes. GPU, CUDA, driver and package versions influence results. No model pruning is claimed in this experiment.

9. Future work

Evaluate TensorRT FP16 on a compatible NVIDIA GPU, INT8 quantization with representative calibration data, structured pruning followed by fine-tuning, and profiling of preprocessing, inference, tracking and rendering separately.

10. Conclusion

Complete after benchmarking. State the measured speed improvement, quality change, selected configuration and the exact hardware/software conditions.